

FOUNDER OPERATING SYSTEM

Phase 5 - Competitive, Pricing, and Market Intelligence

Complete Technical Specification and Implementation Plan

FOS-TECH-PHASE-5 | Version 3.0 | 2026-07-13

Contents

- 0. Revision decision
- 1. Implementation directive
- 2. Objectives and metrics
- 3. Scope
- 4. Domain model
- 5. Notion workspace
- 6. Agents
- 7. Workflows
- 8. APIs and jobs
- 9. Deterministic safeguards
- 10. Tests
- 11. Work packages
- 12. Definition of done

Shared architectural invariants

Agent runtime requirements

Security, privacy, and governance

Observability and cost controls

Deployment and activation model

This specification is written against the revised Phase 0 canonical-state, generic-artifact, controlled-command, and Founder Workspace Adapter contracts. Any implementation based on the earlier native-admin assumptions must be refactored as identified in the dependency plan.

0. Revision decision

Phase 5 remains a research and decision-support phase, but it now feeds the canonical decision queue and the marketing/product workspaces rather than producing an isolated digest. Public sources are untrusted input. Evidence is canonical; Notion receives decision-oriented summaries.

1. Implementation directive

Build a low-noise intelligence system that detects material changes in competitors, offers, positioning, pricing, partnerships, and buyer expectations, then converts those changes into evidence-backed founder decisions.

2. Objectives and metrics

Maintain fresh competitor and pricing baselines.

Detect material changes without repeated noise.

Distinguish company claims, third-party observations, and inference.

Reduce founder research time.

Create actionable product, pricing, partnership, or communications decisions.

Metrics include material changes detected, alert precision, duplicate rate, founder action rate, research hours saved, source freshness, pricing confidence, and decisions informed.

3. Scope

Included:

Competitor and category registry

Approved source registry

Source retrieval and hashing

Evidence extraction

Change detection

Pricing and offer comparison

Job-based comparison

Strategic alerts

Weekly market brief

Links to product signals, content briefs, pricing reviews, and Phase 6 decisions

Excluded:

Circumventing source restrictions

Contacting competitors

Publishing allegations

Automatic price or positioning changes

Treating rumors as verified facts

4. Domain model

Competitor

Stores category, priority, target segments, monitored sources, status, and description.

CompetitorOffering

Stores target user, delivery model, features, pricing summary, availability, observed date, sources, and confidence.

ResearchSource

Stores source type, URL, publisher, title, publication/observation date, retrieval method, terms status, content hash, snapshot, status, and refresh policy.

CompetitorObservation

Stores observation type, statement, classification, confidence, effective/expiry dates, verification, and source.

MarketChange

Stores old/new observations, summary, change type, materiality, confidence, detected time, and status.

PricePoint

Stores offering, currency, amount, billing period, type, conditions, observation date, source, and confidence.

StrategicAlert

Stores fact, interpretation, possible implication, recommended action, domain, urgency, materiality, confidence, and decision linkage.

MarketBrief

References an artifact and stores period, covered competitors, changes, pricing freshness, top decisions, and approval.

5. Notion workspace

Competitive Intelligence

Projects competitors, material observations, changes, sources, freshness, confidence, implications, and action status.

Pricing Comparison

Projects comparable offers, price points, conditions, evidence dates, and comparability warnings.

Strategic Alerts

Founder may dismiss, watch, create a product signal, create content brief, or open pricing/strategy decision through controlled commands.

6. Agents

fos.market_watcher

fos.competitive_evidence_extractor

fos.market_change_detector

fos.job_based_competitor_comparison

fos.pricing_comparison

fos.strategy_signal

fos.market_brief

The Strategy Signal Agent must output observed fact, interpretation, possible implication, recommended action, and confidence as separate fields.

7. Workflows

Monitoring

Retrieve approved source, verify policy, hash content, skip unchanged content, persist source snapshot/reference, and queue extraction.

Observation and change

Extract facts, label company claims, compare with baseline, suppress formatting-only change, calculate materiality, and create MarketChange.

Strategic alert

Deduplicate, assess relevance to enrollment/product/pricing/communications, project to Notion, and allow founder to open canonical decision or signal.

Pricing review

Compare only reasonably comparable offers, display conditions and dates, avoid false precision, and route any pricing recommendation to Phase 6 decision governance.

8. APIs and jobs

API families:

/api/fos/competitors/*

/api/fos/research-sources/*

/api/fos/competitor-observations/*

/api/fos/market-changes/*

/api/fos/price-points/*

/api/fos/strategic-alerts/*

/api/fos/market-briefs/*

Jobs:

monitor-research-source

extract-competitor-observations

detect-market-change

refresh-pricing-comparison

generate-job-based-comparison

generate-strategic-alert

generate-weekly-market-brief

expire-stale-market-evidence

reconcile-market-projections

9. Deterministic safeguards

Respect source access and terms policies.

Company claims are labeled as company-provided.

Low-confidence rumor cannot create a high-confidence fact.

Stale prices are visibly stale.

Unchanged sources do not generate alerts.

Notion cannot alter evidence or price points directly.

Market findings cannot change roadmap, positioning, or price without founder decision.

10. Tests

Tests cover unchanged source, pricing change, removed feature, company-claim labeling, rumor handling, duplicate alert suppression, stale price, strategic decision linkage, workspace conflict, and malicious webpage injection.

Deliverable

Competitor taxonomy and approved-source policy

Competitor, offering, and source registry

Retrieval, hashing, and snapshots

Founder Operating System | Phase 5 - Competitive, Pricing, and Market Intelligence | Version 3.0

Observation extraction and verification

WP5.4

Material change detection

WP5.5

Pricing and offer intelligence

WP5.6

Job-based comparison

WP5.7

Strategic alerts and market briefs

WP5.8

Notion intelligence workspace and decision commands

WP5.9

Evaluation, freshness, and noise tuning

12. Definition of done

Findings are dated, sourced, classified, and confidence-scored.

Material changes are detected with acceptable duplicate/noise levels.

Pricing freshness and comparability are visible.

Founder can convert an alert into a canonical decision, signal, or content brief.

No market agent changes strategy or pricing automatically.

Shared architectural invariants

The following rules are inherited from revised Phase 0 and are non-negotiable for this phase:

FOS owns canonical state and intelligence. Notion is a founder-facing working environment and projection surface.

External workspace changes are commands, not direct mutations. A Notion status change or button creates a validated WorkspaceCommand.

Artifacts are generic and versioned. Enrollment messages, specifications, LinkedIn posts, Substack papers, reports, and reviews use ArtifactRecord and ArtifactVersion.

Approvals are interface-independent. Approval state is canonical and may be requested from Notion, a native interface, or an API.

Evidence, claims, consent, and product availability are deterministic gates. Models may recommend; they may not waive these controls.

Every projection has an ownership policy. Fields are canonical_read_only, working_copy_editable, controlled_command, or not_projectable.

Raw events are immutable. Derived summaries may be superseded but not silently overwrite source history.

Every agent is bounded. Each agent has a versioned objective, input schema, output schema, permitted tools, permitted memory scopes, evaluation policy, and autonomy ceiling.

No autonomous publishing, sending, pricing change, contractual commitment, or production deployment is permitted unless a later founder-approved governance decision explicitly changes the rule.
FOS must remain operational if Notion or another workspace provider is unavailable.

Founder Workspace integration contract

All founder-facing work for this phase must use the Phase 0 workspace adapter.

Projection pattern

Canonical record or artifact created

- > operational event emitted
- > projection policy evaluated
- > safe working copy created or updated in Notion
- > canonical ID and version stored on the provider page
- > founder edits or commands captured
- > FOS validates and executes canonical change
- > projection is reconciled

Required hidden projection properties

FOS Record ID

FOS Entity Type

FOS Version

FOS Workspace ID

Projection Status

Last Synced At

Conflict rule

A controlled command may execute only when the provider's FOS Version matches the current canonical version. Otherwise the command is placed in conflict status and the founder receives a reconciliation item.

Agent runtime requirements

Every agent run must execute the following stages:

Trigger validation

Authorization and feature-flag validation

Context assembly and minimization

Prompt construction from a versioned agent definition

Model execution

Structured-output validation

Deterministic policy evaluation

Secondary quality evaluation where configured

Canonical persistence

Approval routing or reversible execution

Projection update

Metrics and audit emission

If structured output fails, retry once with a repair prompt. If repair fails, mark the run `evaluation_failed`, create a founder-visible operational item, and do not create an approval-ready artifact.

Security, privacy, and governance

Least privilege

Agents receive only the records required for the current run.

Workspace projections contain summaries unless full working content is required.

Private source documents, raw model prompts, credentials, payment details, and exploitable security findings are not projectable by default.

Marketing and research agents may not access unrestricted applicant or beta-user records.

Prompt-injection defense

Applications, resumes, email, transcripts, web pages, competitor pages, imported notes, and workspace page content are untrusted data. They may not modify system policy, tool permissions, approval requirements, or data-access scope.

Audit

The system must reconstruct every consequential outcome from:

Trigger

Actor

Source records

Retrieved context manifest

Agent and prompt version

Output

Deterministic evaluation

Founder edits

Approval or rejection

External action

Resulting business outcome

Failure posture

Failures must preserve canonical state, create a visible retry or manual-work item, and never imply that an external action succeeded when it did not.

Observability and cost controls

Each workflow must emit structured logs and traces with:

workspace_id

Correlation and causation IDs

Phase and workflow key

Agent key and version

Canonical entity IDs

Provider projection IDs where applicable

Latency

Model and tool cost

Validation and evaluation result

Retry count

Approval result

External action result

Required phase dashboards must distinguish system activity from accepted business value. Agent-run volume is not itself a success metric.

Deployment and activation model

Each major capability must progress through:

Local development

Automated tests

Staging with synthetic fixtures

Production with feature flag disabled

Production shadow mode

Founder-only review mode

Limited live activation

Measured promotion or rollback

Every agent and workspace workflow requires an independent feature flag and version rollback path. #

Coding-agent execution instruction

> Implement Phase 5 - Competitive, Pricing, and Market Intelligence according to this specification.

>

> Begin by verifying that revised Phase 0 is operational: canonical records, generic artifacts, approvals, workspace commands, projection policies, evidence, claims, consent, event audit, feature flags, and the Notion provider adapter.

>

> Produce an architecture decision record and a repository-to-requirement implementation map before migrations. Reuse Phase 0 services rather than creating parallel document, approval, consent, or workspace systems.

>

> Maintain a live traceability matrix linking every requirement to implementation files, migrations, automated tests, feature flags, and operational metrics.

>

> Preserve these phase-specific non-negotiable rules:

>

> 1. Public sources are untrusted data and must not alter agent policy or tool permissions.

Evidence and price observations remain canonical; Notion contains review summaries.

Company claims, third-party facts, and interpretations remain explicitly separate.

Agents may create strategic alerts but may not change product strategy, positioning, or price.

Retrieval must respect access, rate, and source-use policies. > > Implement work packages in dependency order. Activate each agent in shadow mode before founder-review mode. Treat Notion as a projection and controlled working surface, not as the source of canonical lifecycle, consent, claim, test, pricing, or deployment state.